

```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)
    stop("pH_super approximation relative error exceeds 10%")
  z1_frac <- z1_q
}

```

```

    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

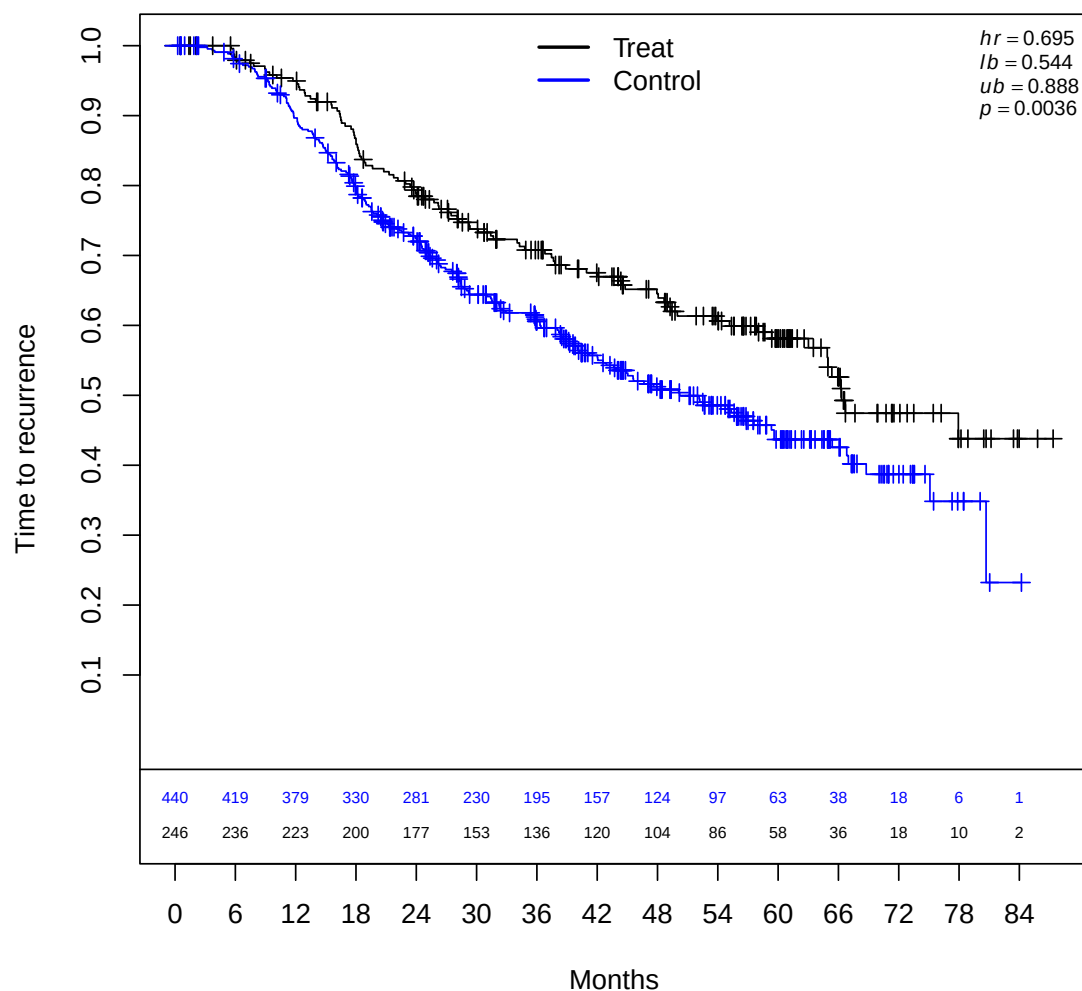
maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

mod.harm <- "alt"
hrH.target <- 2.5
# out.loc = NULL turns off file creation
N <- 700
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



```
## Super-population empirical harm and non-harm hazard ratios= 2.499996 0.6466405
## Causal HR (empirical ITT)= 0.7125876
```

```
file_out <- c("output/sim-FS4_Lasso_ex9.Rdata")

n_add_noise <- 3

# sim=69 sim=71 sim <- 101

sim <- 93

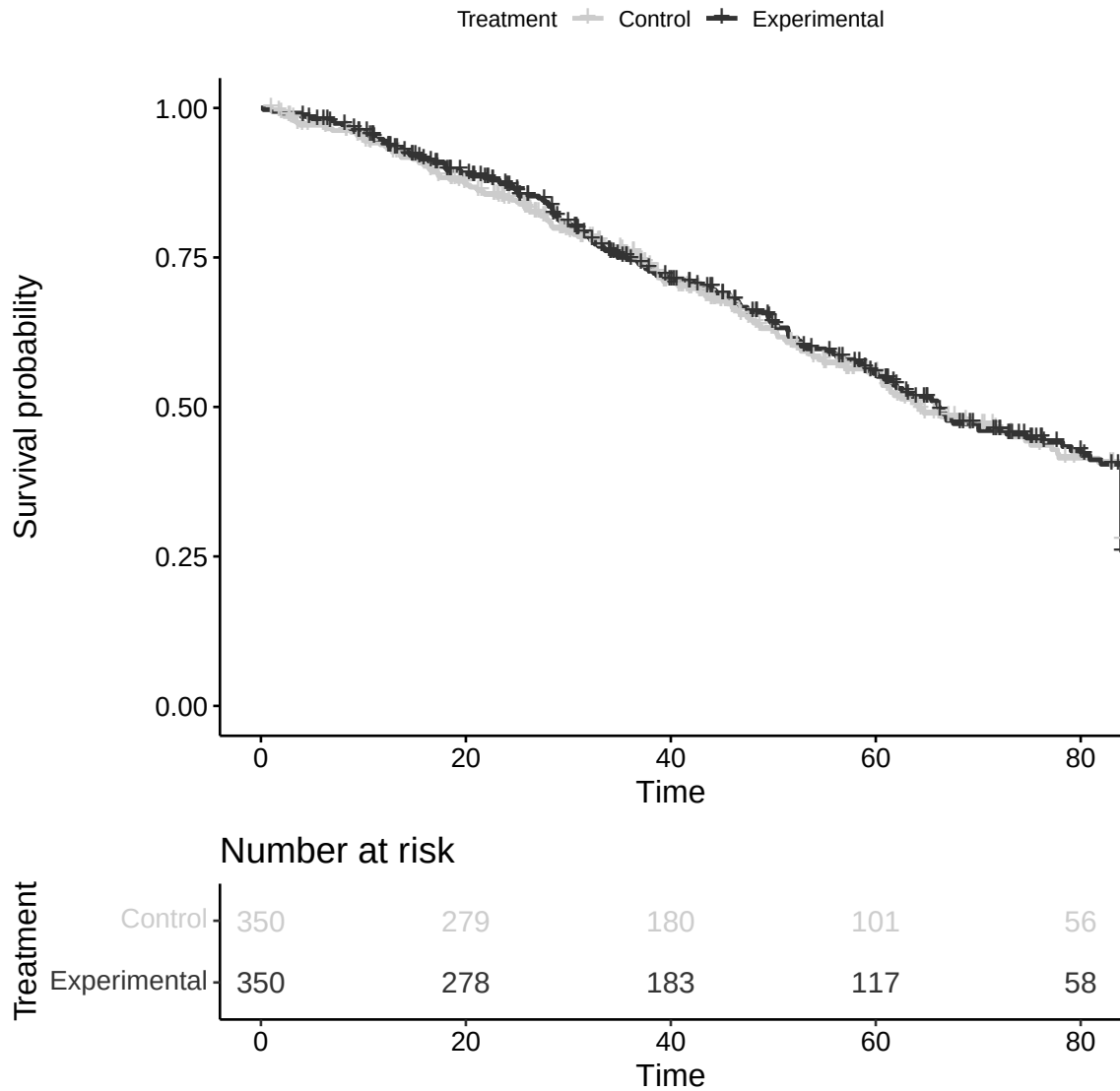
dgm <- this.dgm$dgm

x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
```

```
legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
risk.table = TRUE, risk.table.col = "strata")
```



```
if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
  x$noise2 <- rnorm(N)
  x$noise3 <- rnorm(N)
  x$noise4 <- rnorm(N)
  x$noise5 <- rnorm(N)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
    "noise2", "noise3", "noise4", "noise5")
}
```

```

}

if (n_add_noise == 3) {
  set.seed(8316951 + 1000 * sim)
  # Add 3 noise
  x$noise1 <- rnorm(N, sd = 1)
  x$noise2 <- rnorm(N, sd = 1)
  x$noise3 <- rnorm(N, sd = 1)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
    "noise2", "noise3")
}

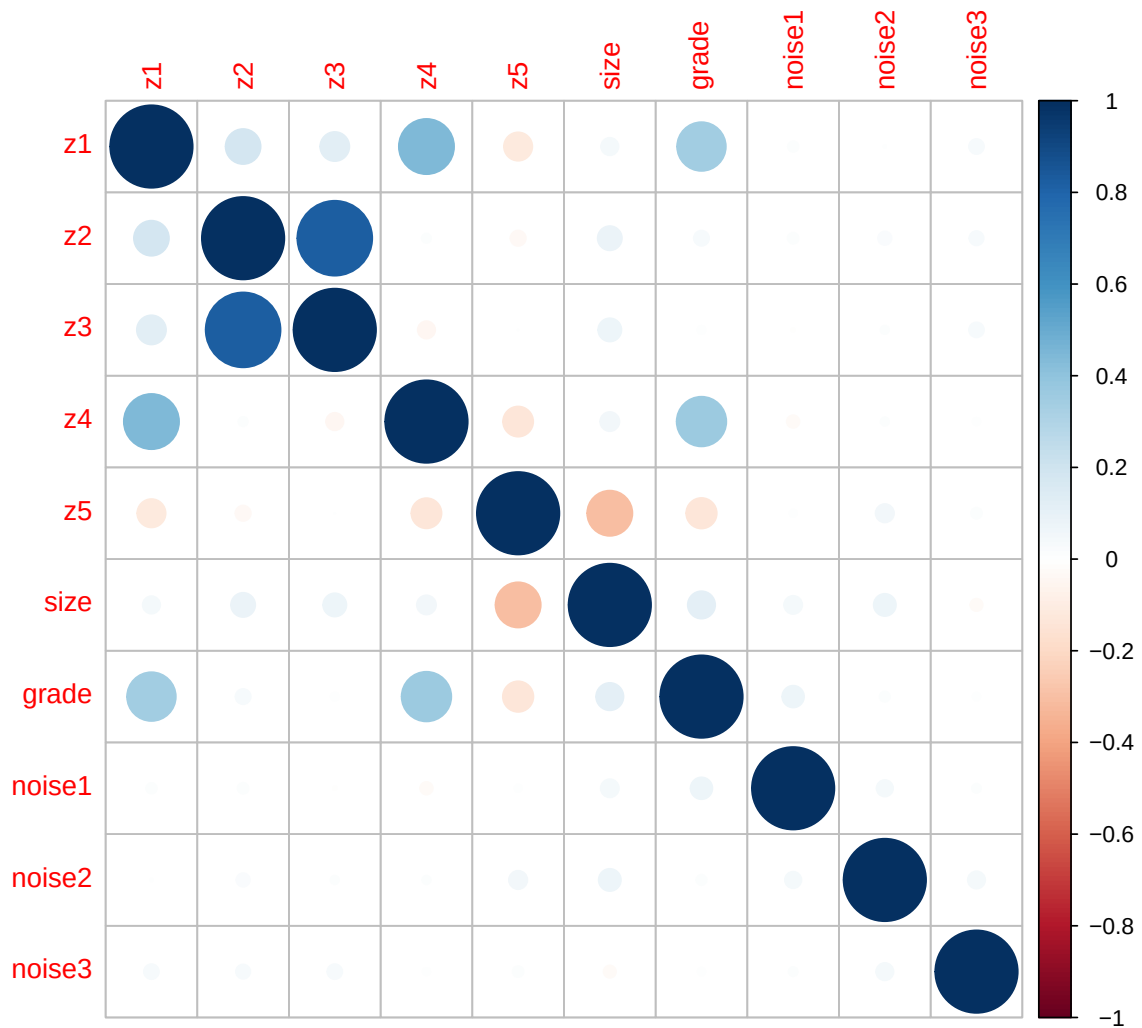
# More challenging which_replace <- which(confounders.name=='z1')
# confounders.name[which_replace] <- 'er'

cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5+size+grade+noise1+
coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef) se(coef)      z      p
## treat  -0.084033  0.919401  0.104156 -0.807  0.420
## v11      0.583313  1.791966  0.128350  4.545 5.50e-06
## v21     -0.245157  0.782582  0.187831 -1.305  0.192
## v31      0.114718  1.121557  0.189435  0.606  0.545
## v41      0.619540  1.858073  0.125825  4.924 8.49e-07
## v51     -0.966706  0.380334  0.111143 -8.698 < 2e-16
## size      0.005259  1.005273  0.003400  1.547  0.122
## grade   -0.172851  0.841263  0.099072 -1.745  0.081
## noise1   0.012392  1.012469  0.051176  0.242  0.809
## noise2   0.085960  1.089762  0.053243  1.614  0.106
## noise3   0.058816  1.060580  0.051580  1.140  0.254
##
## Likelihood ratio test=171.5 on 11 df, p=< 2.2e-16
## n= 700, number of events= 380

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
# Options Allconfounders.name is list of confounders within analysis dataset
# (1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
# Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
# Lasso used to reduce dimension Continuous covariates are cut at medians
# However, if GRF selects a covariate cut then only that cut is used: For
# example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
# <= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
# use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
# F & use_grf =T Median cuts (unless selected via GRF) as in (2) However no
# possible dimension reduction via lasso All categorical factors included

use_lasso <- TRUE
use_grf <- TRUE
use_grf_only <- FALSE

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
  frac.tau = 0.6, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
```

```

event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
pstop.futile = pstop.futile, pconsistency.threshold = pconsistency.threshold,
stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
plot.sg = TRUE, vi.grf.min = vi.grf.min)

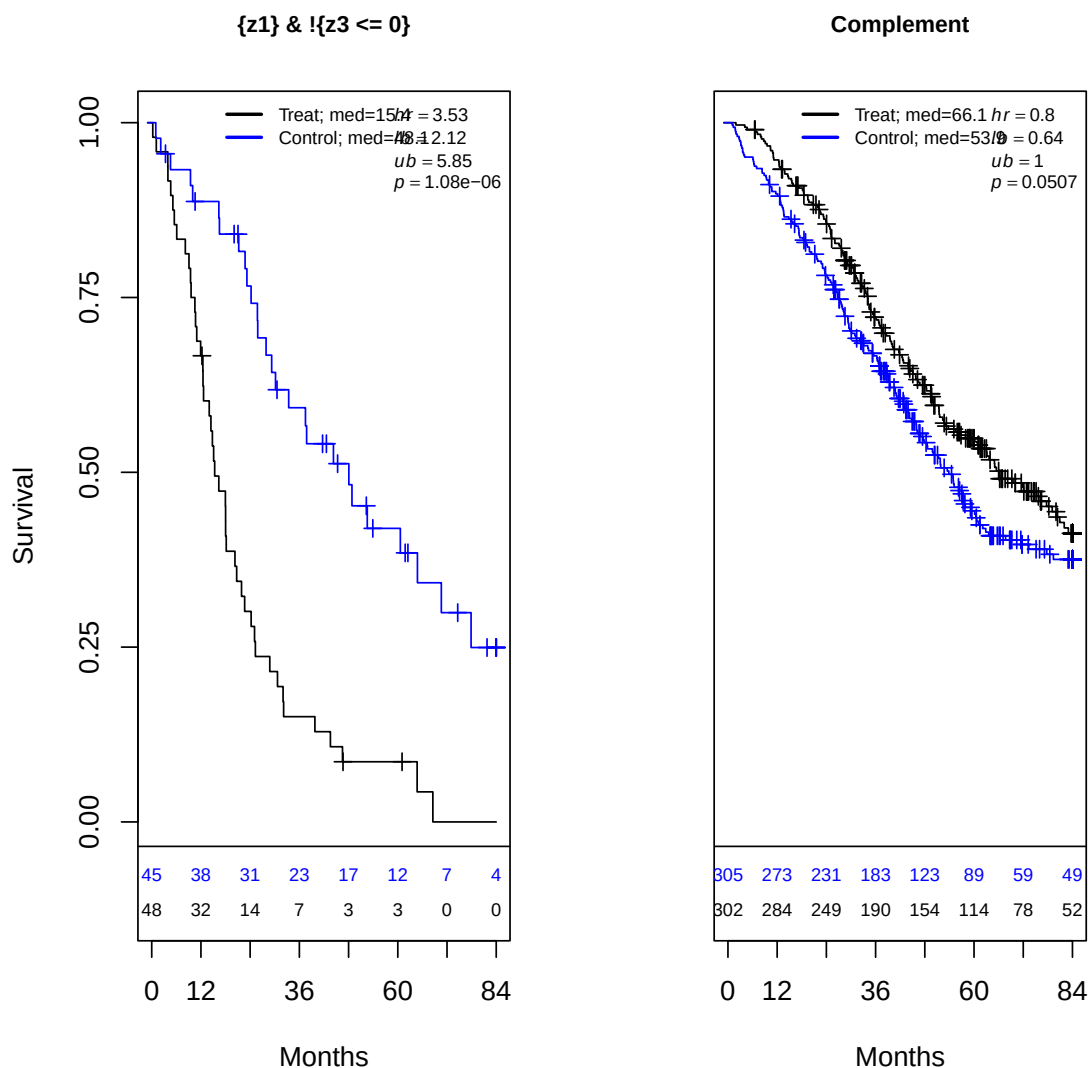
## tau, maxdepth= 47.61047 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2   -3.586075   394.000000   1.395653    3.586075   394.000000
## 2         3    3.170479   306.000000   1.491105   -3.170479   306.000000
## 3         4    3.255032   106.000000   2.273193   -3.255032   106.000000
## 4         5   -3.869056   424.000000   1.275091    3.869056   424.000000
## 5         6   -7.577072    77.000000   3.227939    7.577072    77.000000
## 6         7   15.442303    93.000000   2.923030   -15.442303    93.000000
##   treated.se      diff Nsg depth
## 1   1.395653  -7.172149 394     1
## 2   1.491105   6.340958 306     1
## 3   2.273193   6.510065 106     2
## 4   1.275091  -7.738112 424     2
## 5   3.227939 -15.154145  77     2
## 6   2.923030  30.884605  93     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 6         7    15.44230    93.00000    2.92303   -15.44230    93.00000
##   treated.se      diff Nsg depth
## 6   2.92303 30.88461  93     2
## [1] "z1 <= 0"      "size <= 18" "z3 <= 0"
## [1] "z3 <= 0"
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade
## 10 x 1 sparse Matrix of class "dgCMatrix"
##               s0
## z1      0.541910577
## z2     -0.104495197
## z3      .
## z4      0.566284735
## z5     -0.929614993
## size    0.004222463
## grade  -0.111880991
## noise1  .
## noise2  0.069318372
## noise3  0.039942522
## Cox-LASSO selected: z1 z2 z4 z5 size grade noise2 noise3
## Cox-LASSO not selected: z3 noise1
## Median cuts (after Lasso if use_lasso) size noise2 noise3
## Categorical (after Lasso if use_lasso) z1 z2 z4 z5 grade
## Prior to removing (if in GRF): size noise2 noise3
## Left (to cut at median) after removing others if in GRF: noise2 noise3
## Initial GRF cuts included z1 <= 0 size <= 18 z3 <= 0
## # of candidate subgroup factors= 9
## [1] "noise2 <= median(noise2)" "noise3 <= median(noise3)"
## [3] "size <= 18"                "z1"
## [5] "z2"                        "z4"
## [7] "z5"                        "grade"
## [9] "z3 <= 0"
## Confounders per grf screening q4 q9 q5 q2 q8 q3 q7 q1 q6

```

```

## FSconfounders.name      vi.cs  vi_ratio
## 4                      q4 0.21352744 1.9217469
## 9                      q9 0.20288400 1.8259560
## 5                      q5 0.10757871 0.9682084
## 2                      q2 0.10576234 0.9518610
## 8                      q8 0.09750818 0.8775736
## 3                      q3 0.07757032 0.6981329
## 7                      q7 0.06827248 0.6144523
## 1                      q1 0.06820228 0.6138205
## 6                      q6 0.05869426 0.5282483
## Number of unique levels (L) and possible subgroups= 19 524287
## # of subgroups based on # variables > k.max and excluded (per million) 0.524097
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 21 23
## # of subgroups with sample size less than criteria 33
## # of subgroups meeting all criteria = 156
## # of subgroups fitted (Cox model estimable) = 156
## *Subgroup Searching Minutes=* 0.05098333
## Number of subgroups meeting HR threshold 25
## # of candidate subgroups (meeting HR criteria) = 25
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K   n   E d1   m1   m0   HR L(HR) U(HR) q4.0 q4.1 q9.0 q9.1 q5.0 q5.1
## 1: 2 93 72 45 15.35 48.07 3.53 2.12 5.85 0 1 1 0 0 0
## 2: 2 80 26 15 76.14 NA 2.13 0.97 4.64 0 0 0 0 0 0
## 3: 2 116 86 53 18.17 44.31 2.12 1.36 3.29 0 1 0 0 0 1
## 4: 2 63 25 15 69.51 NA 2.04 0.91 4.55 0 0 0 0 0 1
## 5: 2 79 52 29 28.79 60.67 2.00 1.15 3.46 0 0 1 0 0 0
## 6: 2 64 29 17 40.31 NA 1.86 0.89 3.91 0 0 0 0 0 0
## 7: 2 82 54 32 21.82 37.77 1.63 0.95 2.81 0 1 0 0 0 0
## 8: 2 150 100 56 26.04 46.20 1.63 1.10 2.42 0 0 1 0 0 0
## 9: 2 106 44 25 63.05 NA 1.59 0.88 2.89 1 0 0 0 0 0
## 10: 2 154 80 43 46.52 73.34 1.55 1.00 2.41 0 0 1 0 0 0
##      q2.0 q2.1 q8.1 q8.2 q8.3 q3.0 q3.1 q7.0 q7.1 q1.0 q1.1 q6.0 q6.1
## 1: 0 0 0 0 0 0 0 0 0 0 0 0 0
## 2: 0 0 0 0 0 0 0 1 0 0 0 0 1
## 3: 0 0 0 0 0 0 0 0 0 0 0 0 0
## 4: 0 0 0 0 0 0 0 1 0 0 0 0 0
## 5: 0 0 0 0 0 1 0 0 0 0 0 0 0
## 6: 0 0 0 0 0 0 0 1 0 0 0 1 0
## 7: 0 0 0 0 1 0 0 0 0 0 0 0 0
## 8: 0 0 0 0 0 0 0 0 0 0 0 0 1
## 9: 0 0 0 0 0 0 0 1 0 0 0 0 0
## 10: 0 1 0 0 0 0 0 0 0 0 0 0 0
## Consistency 1
## # of splits= 400
## Model, % Consistency Met= {z1} !{z3 <= 0} 1
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K   M.1      M.2
## 1: 1 93 1 1 2 {z1} !{z3 <= 0}

```



```
## [1] "{z1}"      "!("{z3 <= 0}"
## % consistency criteria met= 1
## SG focus= hr
## Subgroup Consistency Minutes= 0.02301667
## Subgroup found
## Minutes overall= 0.09726667

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 79.35078 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2   -6.937012   394.000000   2.614575    6.937012   394.000000
## 2         3    4.396885   306.000000   2.776842   -4.396885   306.000000
## 3         4   -7.791878   422.000000   2.446217    7.791878   422.000000
## 4         5    7.039671   108.000000   4.852437   -7.039671   108.000000
## 5         6  -15.532784    77.000000   5.524103   15.532784    77.000000
## 6         7   25.120178    93.000000   4.599323  -25.120178    93.000000
```

```
##   treated.se      diff Nsg depth
## 1   2.614575 -13.87402 394     1
## 2   2.776842   8.79377 306     1
## 3   2.446217 -15.58376 422     2
## 4   4.852437  14.07934 108     2
## 5   5.524103 -31.06557  77     2
## 6   4.599323  50.24036  93     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 6         7      25.120178    93.000000   4.599323  -25.120178    93.000000
##   treated.se      diff Nsg depth
## 6   4.599323 50.24036  93     2
## [1] "z1 <= 0"          "noise2 <= 0.81" "z3 <= 0"
## [1] "z3 <= 0"
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_mat")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")

## Minutes for Boots 1000 5.4904

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
```

```

## Projection per 1000 5.4904

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")

## Propn bootstrap subgroups found = 0.996

# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmns_search > max.minutes)),
    "\n")

## Number timed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 3.525673 0.9110191 2.124687 5.850445 3.059576 0.9031706 1.715501 5.456717

print(Hc_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 0.7999822 0.09138446 0.6395064 1.000727 0.813209 0.140371 0.5797933 1.140594

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")

## Overall minutes for analysis 5.665433

save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

# dfH <- subset(fs.est$df.est, flag.harm==1) dfHc <- subset(fs.est$df.est,
# flag.harm==0) dfH_est <- subset(fs.est$df.est, treat.recommend==0) dfHc_est
# <- subset(fs.est$df.est, treat.recommend==1)

# dfSens <- subset(fs.est$df.est, flag.harm==1 & treat.recommend==0) sensH <-
# nrow(dfSens)/nrow(dfH)

## Confounders evaluated in Forestsearch:
## [1] "noise2 <= median(noise2)" "noise3 <= median(noise3)"
## [3] "size <= 18"                "z1"
## [5] "z2"                        "z4"
## [7] "z5"                        "grade"
## [9] "z3 <= 0"

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT					
ITT	0.966	0.79	1.181	700	.
H subgroup estimates					
H_{oracle}	3.526	2.125	5.851	93	.
$\hat{H}$	3.526	2.125	5.851	93	93
$\hat{H}_{bc}$	3.06	1.716	5.456	93	93
H-complement subgroup estimates					
H_{oracle}^c	0.8	0.64	1	607	.
$\hat{H}^c$	0.8	0.64	1	607	607
$\hat{H}_{bc}^c$	0.813	0.58	1.139	607	607